

EXPLICACIÓ LAB1 - DAVID PÉREZ CRUZ

CODI TT.C:

En aquesta part del codi el que fem és simplement comprovar que el número de caràcters sigui igual a 3 (sino mostra un error).

Validem que els números passats al programa siguin enters majors a 0, utilitzant la funció "nombre_valid" y si son correctes, els hi assignem els valor de M y de N.

```
int nombre_valid(char *str) {
    int num = atoi(str);
    return num > 0;
}

int main(int argc, char *argv[]) {

    if (argc != 3) { // Comprovem que únicament es passin 3 paràmetres
        printf("Ús: ./tt M > 0 N > 0 \n");
        exit(-1);
    }

    if (!nombre_valid(argv[1]) || !nombre_valid(argv[2])) {
        // Verificació d'enters positius
        printf("Error: M i N han de ser enters més grans que 0\n");
        exit(-1);
    }

    int M = atoi(argv[1]);
    int N = atoi(argv[2]);
}
```

Tot seguit, el que fem es crear els replicants, creem un array per guardar els codis de sortida per després mostrar-los.

Fem un bucle per a tots els replicants, on a pid fem la clonació utilitzant fork().

Si el pid es igual a 0, significa que es un fill y per tant executem “./replicant”, si dona error sortim amb un “exit(-1)”;

Si el pid es major a 0, significa que es el pare, llavors guardem el PID del replicant al array creat anteriorment.

Si el pid no es ni igual a 0, ni major, significa que ha donat un error en el fork i fem “exit(-1)”.

```
// Crear tots els replicants
int exit_codes[M]; // Array per guardar els codis de sortida
for (int i = 0; i < M; i++) {
    int pid = fork(); // Clonació

    if (pid == 0) { // Mutació per al fill
        execlp("./replicant", "./replicant", argv[2], (char*)0);
        perror("Error exec replicant"); // Si execlp falla
        exit(-1);
    } else if (pid > 0) { // El pare continua
        exit_codes[i] = pid; // Guardar el PID del replicant
    } else { // Si fork falla
        perror("Error fork");
        exit(-1);
    }
}
```

Per finalitzar, el que fem és un altre bucle per a esperar que cada replicant acabi amb “wait” i imprimir el resultat del stat d'èxit.

```
// Esperar a que tots els replicants acabin i imprimir els codis de
sortida
for (int i = 0; i < M; i++) {
    int stat;
    int pid = wait(&stat); // Espera que cada replicant acabi
    if (WIFEXITED(stat)) {
        exit_codes[i] = WEXITSTATUS(stat); // Guardar el codi de
sortida real
        printf("--> Exit code [%d]\n", exit_codes[i]);
    }
}

printf("Done!\n");
exit(0);
```

CODI REPLICANT.C:

Al codi del replicant, primer de tot incloem “dummy.h”.

Acte seguit, el que fem és crear un buffer de 200 caràcters i comprovar que N sigui menor a 200, si no ho és imprimeix un error i surt amb un “exit(-1)”;

Si sí que ho és, inicialitzem la variable “final” a 0, inicialitzem el dummy, i fem un bucle mentre que final valgui 0.

Lavors final agafa el resultat de la funció “dummy_cpt”. I fem un “dummy_end()”.

Si tot ha sigut exitós, sortim amb un “exit(0);

```
#include "dummy.h"
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {

    int N = atoi(argv[1]);
    char buffer[200];

    if (N < 200) {
        int final = 0;
        dummy_ini( buffer, N );
        while (!final) {
            final = dummy_cpt( buffer, N );
        }
        dummy_end();
    } else {
        printf("Error: N excedeix la mida del buffer.\n");
        exit(-1);
    }

    exit(0);
}
```

COMPROVACIÓ DEL CODI:

```
david@david-Modern-14-C13M:~/2no/SIOP/lab1$ gcc tt.c -o tt -L. -ldummy
david@david-Modern-14-C13M:~/2no/SIOP/lab1$ gcc replicant.c -o replicant -L. -ldummy
david@david-Modern-14-C13M:~/2no/SIOP/lab1$ ls -l
total 52
-rw-rw-r-- 1 david david  91 sep  9 18:24 dummy.h
-rw-rw-r-- 1 david david 3404 sep  9 18:24 libdummy.a
-rw-rw-r-- 1 david david  358 oct  1 13:30 readme.txt
-rwxrwxr-x 1 david david 16288 oct  1 14:26 replicant
-rw-rw-r-- 1 david david  430 oct  1 13:33 replicant.c
-rwxrwxr-x 1 david david 16288 oct  1 14:26 tt
-rw-r--r-- 1 david david  1753 oct  1 14:09 tt.c
david@david-Modern-14-C13M:~/2no/SIOP/lab1$ ./tt 2 3 5
Ús: ./tt M > 0 N > 0
david@david-Modern-14-C13M:~/2no/SIOP/lab1$ ./tt 2
Ús: ./tt M > 0 N > 0
david@david-Modern-14-C13M:~/2no/SIOP/lab1$ ./tt 2 200
Error: N excedeix la mida del buffer.
Error: N excedeix la mida del buffer.
--> Exit code [255]
--> Exit code [255]
Done!
david@david-Modern-14-C13M:~/2no/SIOP/lab1$ ./tt 2 3
*** DUMMY: Init dummy code...
*** DUMMY: Init dummy code...
*** DUMMY: Exit dummy code...
--> Exit code [0]
*** DUMMY: Exit dummy code...
--> Exit code [0]
Done!
```